Design Document

Event Planner's Dashboard

1. Project Overview

The **Event Planner's Dashboard** is designed to help event planners choose the best dates and locations for outdoor events based on real-time weather forecasts and air quality data. The application fetches data from two APIs: one for weather information and another for air quality metrics, allowing users to make informed decisions for event scheduling. The dashboard provides customizable visualizations for weather conditions and air quality levels. Users can save their preferences for future use and access their historical data to compare trends over time.

2. Structure and Key Components

The dashboard is divided into three main sections:

1. **Weather Data Section**: Displays data such as temperature, humidity, and wind speed in line graphs.
2. **Air Quality Data Section**: Visualizes air quality metrics (e.g., PM2.5, PM10) using bar charts.
3. **Map and Preferences Section**: Allows users to select locations from a map and save preferences for easy future access.

Main Components:

- **Location and Date Picker**: A dropdown menu for selecting regions and a date picker for specifying date ranges.
- **Weather Visualizations**: Graphs showing temperature, precipitation, and wind speed over the selected date range.
- **Air Quality Visualizations**: Bar charts representing air quality metrics such as PM2.5 and PM10 levels.
- **User Preferences**: Users can save preferred locations, date ranges, and data filters for easy future access.

3. Components and Interfaces

The project will utilize a **Model-View-Controller (MVC)** architecture to separate data processing, user interaction, and interface rendering. Here are the core components:

1. **Weather API Handler**:
    - Responsible for retrieving weather data from the Finnish Meteorological Institute or form Open Weather.
    - JSON data is parsed and passed to the controller for processing.
2. **Air Quality API Handler**:
    - Fetches air quality metrics (PM2.5, PM10) from Open AQ.

- The API handler parses the JSON response and forwards the data for visualization.

3. **Controller**:
   - Manages the interaction between the user interface and the backend API handlers.
   - Responsible for coordinating API requests and combining weather and air quality data.
4. **User Interface (UI)**:
   - Built using **JavaFX**.
   - Displays data visualizations (line graphs for weather, bar charts for air quality) and handles user interactions such as location selection, date picking, and saving preferences.

Class Diagram:

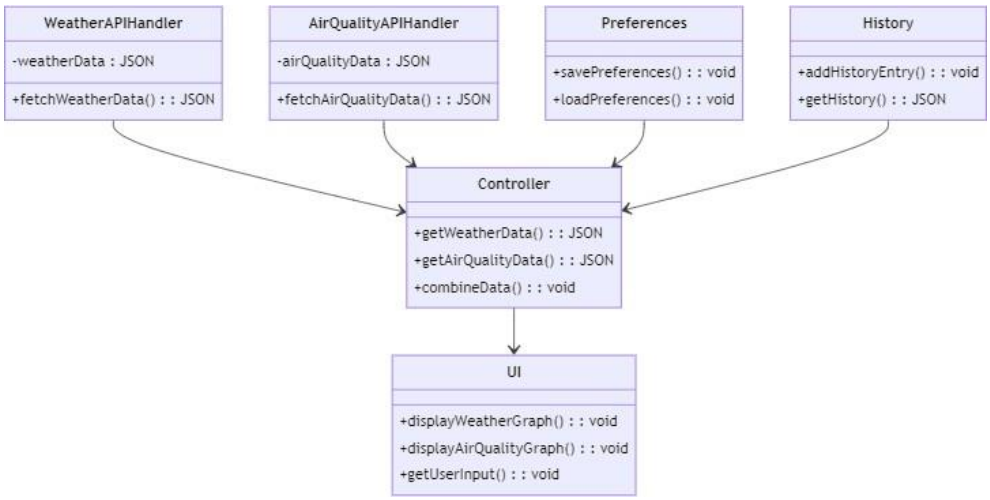


Figure 1 Event Planner's Dashboard Class Diagram

4. Design Patterns and Architecture

Several design patterns and architectural principles are employed for scalability, maintainability, and efficiency:

1. **Model-View-Controller (MVC)**:
   - Separates the backend logic (Model), the user interface (View), and the control flow (Controller). This ensures better maintainability and easier future updates.
2. **Singleton Pattern**:
   - Used for managing API handlers to ensure that only one instance of each API handler is running during the application's lifetime. This helps in optimizing API calls and managing resources.

3. **Factory Pattern**:
    - For creating objects related to API requests and data handling, enabling flexibility in handling different types of requests (weather vs. air quality data).
4. **Observer Pattern (Optional)**:
    - Could be used for real-time updates to the dashboard if a future version needs automatic refresh of data (e.g., live weather updates).

5. APIs and Data

The application will retrieve data from two main APIs:

1. **Weather API**:
    - Finnish Meteorological Institute's Open Data API and Open Weather's API provides real-time weather data, such as temperature, humidity, wind speed, and precipitation forecasts.
    - **Example Endpoint**: GET https://opendata.fmi.fi/weather/forecast?location=
    - **Data Type**: JSON response with forecast details over a selected date range.
2. **Air Quality API**:
    - The Open AQ Platform API provides real-time air quality data, including PM2.5 and PM10 levels.
    - **Example Endpoint**: GET https://api.openaq.org/v1/latest?coordinates=
    - **Data Type**: JSON response with air quality metrics for the selected location.

6. Saved Preferences

The **Saved Preferences** feature allows users to store frequently used configurations, making it easy to reload previously selected locations, date ranges, and metrics without repeating the setup process.

Key Features:

1. **Preference Name**: Users can give a custom name to each saved configuration.
2. **Location and Date Range**: Stores the selected region and date range.
3. **Saved Metrics**: Includes weather and air quality metrics like temperature, PM2.5, and humidity.
4. **Actions**: Users can load, edit, or delete saved preferences.
5. **Quick Overview (Optional)**: Displays a mini-summary or chart preview of the saved data.

Screen Layout:

- **Table View**: A simple table with columns for preference name, location, date range, and saved metrics.

- **Card Layout (Optional)**: Cards showing saved configurations with action buttons (Load, Edit, Delete).

7. History

The **History** section allows users to revisit previous queries, making it easy to compare past weather and air quality conditions with current data.

Key Features:

1. **Timestamp**: Shows when the user last viewed a particular location or preference.
2. **Location and Date Range**: Displays previously selected regions and date ranges.
3. **Weather and Air Quality Summary**: Provides a snapshot of the weather and air quality conditions at the time.
4. **Actions**: Users can view past data or compare multiple historical entries.
5. **Optional Data**: Icons and short summaries of the past conditions for quick reference.

Screen Layout:

- **Timeline View**: A vertical timeline of past entries with timestamps, summaries, and action buttons (View Again, Compare).
- **Table View (Alternative)**: A standard table with columns for timestamp, location, weather, and air quality.

8. Tentative Approaches

In the prototype phase, the following approaches are proposed:

1. **Data Retrieval**: Data will be fetched asynchronously to ensure a smooth user experience.
2. **Visualization**: Simple line and bar graphs will be used for weather and air quality visualizations. Technologies like JavaFX or web-based libraries such as Chart.js will be explored.
3. **Customizability**: Users will be able to customize filters, save preferences, and adjust parameters such as location and date range.
4. **AI Integration (Future)**: As part of future development, AI-based suggestions for optimal event planning days may be explored based on weather and air quality trends.

9. Self-Evaluation

In this section, we evaluate the effectiveness of our initial design and how well it has supported the development process. We follow our initial design plan, assess its potential areas of improvement, and identify any adjustments needed for the implementation.

- Successes:
    - Front-End Design: We applied JavaFx for the front-end design, using SceneBuilder to streamline the creation of view components. This tool enables us to visually design the initial interface and generate corresponding FXML files, which effectively affects the development process. As a result, we successfully created four key views:

        - About view: Provides users with an overview of the application's purpose and features.
        - Dashboard view: Provides users with the primary interface of the application, displaying real-time weather data, calendar and other relevant information.
        - History view: Allows users to view the calendar of plan schedules.
        - SavePreferences view: Enables users to configure their settings.
    - IDE choice: We utilized Apache NetBeans to develop and deploy the application.

- Challenges:
    - FXML and Controller synchronization: While SceneBuilder simplified Front-End design but is difficult to keep FXML files synchronized with JavaFX controllers.

- Implementation Progress:
    - The major views (About, Dashboard, History and SavePreferences) have been implemented according to plan, aligning with the initial structure.
    - Initial data visualizations are functional, providing essential weather and air quality data, but we still considering further enhancements.

G. Conclusion

The **Event Planner's Dashboard** is structured around providing event planners with accurate and useful data to make informed decisions about scheduling outdoor events. The separation of core components, clear user interface, and use of design patterns ensures that the application will be scalable and maintainable for future extensions. The addition of **Saved Preferences** and **History** features enhances the user experience by offering convenient data retrieval and comparison functionality.